UNIVERSIDADE CATÓLICA DE BRASÍLIA
Bacharelado em Engenharia de Software

Relatório Final

Sistema de Controle de Vendas — Toy Storey

Item B8 — Etapa 1 revisada e Etapa 2 consolidadas

Disciplina	Laboratório de Banco de Dados (GPE17M40083)
Professor	Samuel Novais Moura Júnior
Semestre	2026/2
Equipe	José Antônio Rodrigues Tozetti, Henrique Kaio Ferreira da Silva, Gustavo Teixeira de Sousa
SGBD / Linguagem	MySQL 8.0 / Java (JDBC puro, interface Swing)
Repositório	github.com/MammamiaCromia/Toy-Storey
Local e data	Brasília — DF, outubro de 2026

Sobre este documento. O item B8 pede um relatório consolidado que incorpore a Etapa 1 revisada e a Etapa 2, além de uma apresentação de quinze minutos com demonstração da aplicação em execução. Este PDF reúne o relatório (Partes I a III) e, no Apêndice A, o roteiro cronometrado da apresentação. Os trechos que descrevem resultados são rastreáveis aos arquivos do repositório; onde algo foi *verificado novamente* durante a elaboração do relatório, isso vem dito no texto, e o que ainda não está implementado aparece como pendência, sem ser apresentado como pronto.

Sumário

1. Introdução e visão geral **4**

1.1 Como o relatório está organizado 4

1.2 Mapa de entregas no repositório 4

1.3 Como reproduzir o ambiente 4

2. Escopo, regras de negócio e requisitos (A1) **6**

3. Modelo entidade-relacionamento conceitual (A2) **7**

4. Dicionário de dados conforme o DDL (A3) **8**

5. Modelo lógico relacional e normalização (A4 e A5) **11**

5.1 Esquema e políticas de integridade referencial 11

5.2 Escolha de chaves 11

5.3 Verificação de normalização 11

6. Registro de revisões da Etapa 1 **12**

7. Aplicação (B1) **13**

7.1 A operação central: registrar uma venda 13

8. Camada de acesso a dados (B2) **14**

9. Transações e controle de concorrência (B3) **14**

10. Otimização de consultas (B4) **15**

10.1 Discussão 15

11. Segurança e controle de acesso (B5) **17**

11.1 Perfis e princípio do menor privilégio 17

11.2 Visões para restringir colunas e linhas 17

11.3 Armazenamento de senhas 17

11.4 Dados pessoais do projeto e cuidados adotados 17

11.5 Verificação das permissões 18

12. Gatilhos e auditoria (B6) **19**

12.1 Estrutura da trilha 19

12.2 Os dois gatilhos e a justificativa pelo domínio 19

12.3 Evidência de funcionamento 19

12.4 Defeito encontrado: o usuário do banco gravado é sempre o definidor do gatilho 19

13. Projeto físico e armazenamento (B7) 21

13.1 Revisão dos tipos de dados . . . 21

13.2 Estimativa de crescimento em dois anos . . . 21

13.3 Plano de backup . . . 22

13.4 Particionamento e arquivamento histórico . . . 22

14. Avaliação do atendimento aos requisitos e pendências 24

14.1 Requisitos funcionais e não funcionais . . . 24

14.2 Pendências priorizadas . . . 24

15. Conclusão e lições aprendidas 25

16. Uso de inteligência artificial 25

Referências . . . 25

A.1 Demonstração ao vivo (4 min 30 s) . . . 26

A.2 Checklist de preparação (véspera e antes de entrar) . . . 27

A.3 Perguntas prováveis e respostas curtas . . . 27

1. Introdução e visão geral

O projeto Toy Storey modela o ponto de venda de uma loja de brinquedos fictícia: cadastro de clientes e funcionários (generalização de PESSOA), catálogo por categoria, kits promocionais (autorrelacionamento em PRODUTO), vendas com formas de pagamento múltiplas, devoluções e programa de fidelidade por pontos. A Etapa 1 entregou o projeto e a construção do banco; a Etapa 2 acrescentou a aplicação, a camada de acesso, o controle transacional, a otimização de consultas, a segurança, a auditoria e o projeto físico. Este relatório fecha o ciclo (item B8).

1.1 Como o relatório está organizado

Parte	Conteúdo	Itens
I — Etapa 1 revisada	Regras de negócio, modelo conceitual, dicionário de dados conforme o DDL, modelo lógico, normalização e o registro das revisões feitas	A1 a A5
II — Etapa 2	Aplicação, camada de acesso, transações, otimização, segurança e controle de acesso, gatilhos e auditoria, projeto físico	B1 a B7
III — Fechamento	Avaliação do atendimento aos requisitos, pendências, lições aprendidas, uso de IA	B8
Apêndices	A: roteiro da apresentação de 15 minutos. B: correções propostas ao repositório	—

1.2 Mapa de entregas no repositório

Item	Entrega	Onde está
A1–A5	Relatório da Etapa 1 (13 páginas) e documentos de apoio	docs/relatorio-etapa1.pdf, mer-conceitual, dicionario-dados, modelo-logico, normalizacao, regras-negocio
—	Banco: DDL, carga (710 comandos INSERT, 15 consultas de verificação)	sql/01_ddl.sql, 02_carga.sql, 03_consultas.sql
B1	Aplicação Java Swing (10 telas, 3 pacotes)	ToyStoreySystem/src/br/com/{visao, controle, entidade}
B2	Camada de acesso, injeção de SQL, credenciais fora do código	docs/camada_de_acesso.pdf; DAO.java e CRUD*.java
B3	Transações e concorrência	docs/Transacoes_e_Concorrencia.pdf; CRUDCliente, CRUDVenda
B4	Otimização de consultas (3 estudos)	docs/b4_relatorio.pdf; sql/04_volume_b4.sql
B5	Perfis, visões, hash de senha, dados pessoais	sql/05_acesso.sql; Senha.java; Seção 11 deste relatório
B6	Gatilhos e auditoria	sql/01_ddl.sql (tabela auditoria e 2 gatilhos); Seção 12
B7	Projeto físico e armazenamento	Seção 13 deste relatório
B8	Este relatório e a apresentação	Este PDF; Apêndice A

1.3 Como reproduzir o ambiente

A ordem abaixo foi executada do zero durante a elaboração deste relatório, e todos os scripts rodaram sem erro (as 15 consultas de verificação também). Um detalhe importante para a demonstração: a carga `02_carga.sql` grava hashes *fictícios* em `funcionario.senha_hash`, e a classe Senha só aceita o formato `pbkdf2$...`. Por isso, antes do primeiro login é preciso executar `GeradorSenhas`, que regrava o hash de todos os funcionários com uma senha de demonstração.

Passo	Ação
1	Executar `sql/01_ddl.sql` (cria o banco `loja_toystorey`, tabelas, restrições nomeadas, visões, tabela de auditoria e gatilhos)
2	Executar `sql/02_carga.sql` (90 pessoas, 40 categorias, 60 produtos, 73 vendas, 173 itens, 93 pagamentos, 45 devoluções, 46 composições de kit)
3	Executar `sql/05_acesso.sql` (usuários app_gerente, app_operador, app_leitura e app_auth com privilégios mínimos)

Passo	Ação
4	Copiar `config.example.properties` para `config.properties` e preencher as credenciais; executar `GeradorSenhas` uma vez
5	Opcional, para o estudo B4: executar `sql/04_volume_b4.sql` (massa sintética de ~200 mil vendas)

Parte I — Etapa 1 revisada

Escopo, modelo conceitual, dicionário de dados, modelo lógico e normalização, ajustados ao que está de fato no banco

2. Escopo, regras de negócio e requisitos (A1)

O domínio e as 20 regras de negócio (RN01–RN20) da Etapa 1 foram mantidos. A coluna final do quadro é nova: ela registra, para cada regra, **onde ela está realmente garantida hoje** (restrição do banco, código da aplicação) a partir da leitura do DDL e do código Java. Isso substitui a coluna "como é atendida", que descrevia apenas a intenção de projeto.

RN	Regra (resumo)	Prevista	Situação verificada
RN01	Estoque nunca negativo; ajustes manuais com justificativa	Banco + App	Implementado: CHECK (quantidade_estoque >= 0) e UPDATE protegido na venda. Parcial: ajuste manual é auditado (B6), mas sem campo de justificativa
RN02	Código de barras único	Banco	Implementado: UNIQUE uq_produto_codigo_barras
RN03	Mais de uma forma de pagamento por venda	Banco	Parcial: o modelo permite N pagamentos; a tela registra uma forma por venda
RN04	Desconto até 10% para operador; acima exige gerente	App	Pendente: a aplicação não aplica desconto
RN05	Inativar, não excluir, produtos vendidos e clientes com histórico	Banco + App	Parcial: produto e funcionário têm inativação e FK RESTRICT; cliente tem inativação, mas também exclusão física, e fk_venda_cliente usa ON DELETE SET NULL (ver Seção 6, item 9)
RN06	Bloquear venda por classificação etária	App	Implementado em CRUDVenda (idade calculada no banco contra classificacao_idade)
RN07	Congelar o preço no momento da venda	Banco	Implementado: preco_congelado NOT NULL, gravado com o preço vigente
RN08	Baixar os componentes ao vender um kit	App	Implementado: baixarKit() desconta kit_composicao dentro da transação
RN09	Devolução exige a venda original	Banco	Parcial: FK id_venda NOT NULL; ainda não há tela de devolução
RN10	Prazos de troca (7 e 30 dias)	App	Pendente
RN11	Vendedor obrigatório na venda	Banco	Implementado: id_funcionario NOT NULL, preenchido pela sessão
RN12	Alerta de reposição	Consulta	Parcial: consulta 13 lista itens abaixo do mínimo; sem alerta na interface
RN13	Cancelar venda antes do pagamento devolve o estoque	App	Pendente: o status Cancelada existe, mas não há rotina de cancelamento
RN14	Preço mínimo (não vender abaixo do custo)	App	Parcial: ck_produto_precos garante preco_venda > preco_custo no cadastro; sem desconto, não há como violar na venda
RN15	Desconto de aniversário	App	Pendente
RN16	Fechamento cego de caixa	App	Pendente
RN17	Avarias vão para estoque de perdas	Banco + App	Parcial: campo tipo_destino existe; sem estoque separado nem tela
RN18	Cadastro simplificado	Banco	Revisada (Seção 6, item 8): CPF, nome e data de nascimento são obrigatórios
RN19	Reajuste em lote por categoria	App	Parcial: percentual_reajuste_padrao é cadastrado; falta a rotina de UPDATE em lote
RN20	1 ponto a cada R$ 100,00; resgate a partir de 10 pontos	App	Parcial, com divergência: a aplicação credita 1 ponto por R$ 1,00 (comentário no código diz "suposição"); resgate não implementado

Quadro 1 — Rastreabilidade das regras de negócio. Verde: implementado; amarelo: parcial; vermelho: pendente.

Requisitos funcionais e não funcionais são avaliados na Seção 14, depois de apresentada a aplicação.

3. Modelo entidade-relacionamento conceitual (A2)

O diagrama abaixo (notação de engenharia da informação) é o da Etapa 1, sem alteração estrutural. Ele mostra a especialização disjunta de PESSOA em CLIENTE e FUNCIONARIO, o núcleo VENDA, ITEM_VENDA, PAGAMENTO e DEVOLUCAO, e o autorrelacionamento N:N de PRODUTO por KIT_COMPOSICAO. O atributo senha_hash de FUNCIONARIO, exigido por RNF02, já constava do diagrama, mas faltava no dicionário (corrigido na Seção 4).

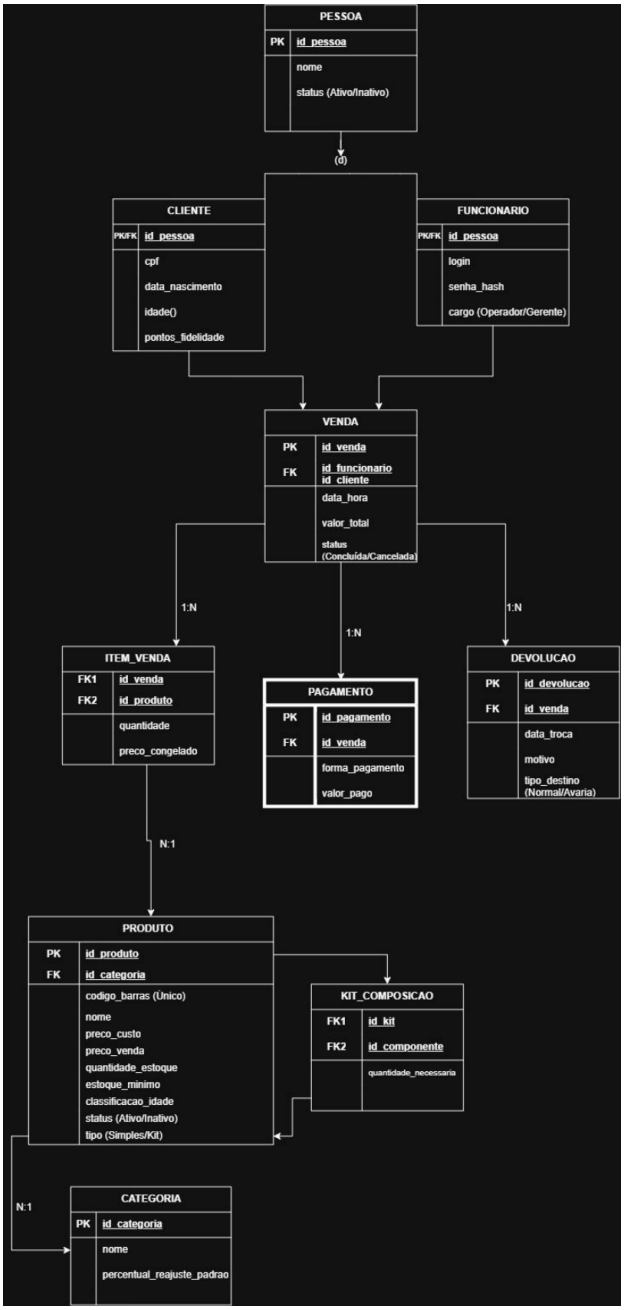


Figura 1 — MER conceitual do Toy Storey (docs/mer-conceitual.pdf).

4. Dicionário de dados conforme o DDL (A3)

Os quadros a seguir foram gerados **a partir do banco criado pelo 01_ddl.sql**, e não copiados do dicionário original; assim, tipos, nulidade e chaves refletem o que o SGBD realmente aplica. Onde isso diverge do dicionário da Etapa 1, a divergência está listada na Seção 6. A coluna "Regra" liga cada atributo às regras de negócio (RN), requisitos (RF/RNF) ou itens de projeto que o justificam.

4.1 PESSOA

Coluna	Tipo	Nulo	Chave	Regra / observação
id_pessoa	INT	Não	PK, AI	
nome	VARCHAR(100)	Não	índice/FK	
status	VARCHAR(20) (padrão Ativo)	Não		RN05 (inativação)

4.2 CLIENTE

Coluna	Tipo	Nulo	Chave	Regra / observação
id_pessoa	INT	Não	PK	
cpf	CHAR(11)	Não	UQ	RN18; dado pessoal (Seção 11.4)
data_nascimento	DATE	Não		RN06, RN15; dado pessoal
pontos_fidelidade	INT (padrão 0)	Não		RN20; CHECK >= 0

4.3 FUNCIONARIO

Coluna	Tipo	Nulo	Chave	Regra / observação
id_pessoa	INT	Não	PK	
login	VARCHAR(100)	Não	UQ	RF02
senha_hash	VARCHAR(255)	Não		RNF02; formato pbkdf2$iterações$sal$hash
cargo	VARCHAR(20)	Não		RF02, RN04; CHECK (Operador, Gerente)

4.4 CATEGORIA

Coluna	Tipo	Nulo	Chave	Regra / observação
id_categoria	INT	Não	PK, AI	
nome	VARCHAR(100)	Não	UQ	UNIQUE
percentual_reajuste_padrao	DECIMAL(5,2)	Sim		RN19

4.5 PRODUTO

Coluna	Tipo	Nulo	Chave	Regra / observação
id_produto	INT	Não	PK, AI	
id_categoria	INT	Não	índice/FK	
codigo_barras	CHAR(13)	Não	UQ	RN02 (UNIQUE)
nome	VARCHAR(150)	Não	índice/FK	
preco_custo	DECIMAL(5,2)	Não		RN14; ver B7
preco_venda	DECIMAL(5,2)	Não		RN19; ver B7
quantidade_estoque	INT (padrão 0)	Não		RN01; CHECK >= 0
estoque_minimo	INT (padrão 0)	Não		RN12, RF04
classificacao_idade	INT (padrão 0)	Não		RN06
status	VARCHAR(20) (padrão Ativo)	Não		RN05
tipo	VARCHAR(20)	Não		RN08 (Simples ou Kit)

4.6 KIT_COMPOSICAO

Coluna	Tipo	Nulo	Chave	Regra / observação
id_kit	INT	Não	PK	
id_componente	INT	Não	PK	
quantidade_necessaria	INT	Não		RN08; CHECK > 0

4.7 VENDA

Coluna	Tipo	Nulo	Chave	Regra / observação
id_venda	INT	Não	PK, AI	
id_funcionario	INT	Não	índice/FK	RN11
id_cliente	INT	Sim	índice/FK	Opcional (venda sem cadastro)
data_hora	DATETIME	Não	índice/FK	
valor_total	DECIMAL(8,2) (padrão 0.00)	Não		Redundância controlada (Seção 5.3)
status	VARCHAR(20) (padrão Concluída)	Não		RN13 (Concluída, Cancelada)

4.8 ITEM_VENDA

Coluna	Tipo	Nulo	Chave	Regra / observação
id_venda	INT	Não	PK	
id_produto	INT	Não	PK	
quantidade	INT	Não		
preco_congelado	DECIMAL(5,2)	Não		RN07; ver B7

4.9 PAGAMENTO

Coluna	Tipo	Nulo	Chave	Regra / observação
id_pagamento	INT	Não	PK, AI	
id_venda	INT	Não	índice/FK	
forma_pagamento	VARCHAR(20)	Não		RN03; CHECK (Dinheiro, Débito, Crédito, Pix)
valor_pago	DECIMAL(8,2)	Não		A soma fecha com valor_total

4.10 DEVOLUCAO

Coluna	Tipo	Nulo	Chave	Regra / observação
id_devolucao	INT	Não	PK, AI	
id_venda	INT	Não	índice/FK	RN09
data_troca	DATETIME	Não		RN10
motivo	VARCHAR(200)	Não		
tipo_destino	VARCHAR(20)	Não		RN17 (Normal, Avaria)

4.11 AUDITORIA

Coluna	Tipo	Nulo	Chave	Regra / observação
id_auditoria	BIGINT	Não	PK, AI	
tabela	VARCHAR(50)	Não	índice/FK	
id_registro	INT	Não		
campo	VARCHAR(50)	Não		
valor_anterior	VARCHAR(100)	Sim		
valor_posterior	VARCHAR(100)	Sim		
id_funcionario	INT	Sim	índice/FK	B6; preenchido via @id_funcionario_app
usuario_banco	VARCHAR(100)	Não		B6 (ver Seção 12.4)
data_hora	DATETIME (padrão current_timestamp())	Não	índice/FK	

Quadro 2 — Dicionário de dados gerado do banco (information_schema).

5. Modelo lógico relacional e normalização (A4 e A5)

5.1 Esquema e políticas de integridade referencial

O mapeamento do MER é o da Etapa 1 (tabela por subclasse para a especialização de PESSOA; tabelas associativas para ITEM_VENDA e KIT_COMPOSICAO; FKs simples para os relacionamentos 1:N). O quadro lista as chaves estrangeiras com as ações referenciais efetivamente declaradas no DDL, que a Etapa 1 não detalhava.

Chave estrangeira	Referencia	ON DELETE	ON UPDATE	Justificativa
cliente.id_pessoa	pessoa	CASCADE	CASCADE	A subclasse não existe sem a superclasse
funcionario.id_pessoa	pessoa	CASCADE	CASCADE	Idem
produto.id_categoria	categoria	RESTRICT	RESTRICT	Categoria com produtos não se apaga
venda.id_funcionario	funcionario	RESTRICT	CASCADE	RN11: venda sempre tem vendedor
venda.id_cliente	cliente	**SET NULL**	CASCADE	Venda aceita cliente nulo; ver Seção 6, item 9
item_venda.id_venda	venda	CASCADE	CASCADE	Item é parte da venda
item_venda.id_produto	produto	RESTRICT	CASCADE	RN05: produto vendido não se apaga
pagamento.id_venda	venda	CASCADE	CASCADE	Pagamento é parte da venda
devolucao.id_venda	venda	CASCADE	CASCADE	RN09
kit_composicao.id_kit	produto	CASCADE	RESTRICT	Apagar o kit apaga sua composição
kit_composicao.id_componente	produto	RESTRICT	RESTRICT	Componente em uso não se apaga
auditoria.id_funcionario	funcionario	RESTRICT	CASCADE	Trilha de auditoria preserva o autor

Quadro 3 — Ações referenciais declaradas no DDL.

5.2 Escolha de chaves

Mantidas as decisões da Etapa 1: chave substituta (INT AUTO_INCREMENT) em CATEGORIA, PRODUTO, VENDA, PAGAMENTO e DEVOLUCAO, porque não há atributo natural estável; chave natural composta em ITEM_VENDA (id_venda, id_produto) e KIT_COMPOSICAO (id_kit, id_componente); chave compartilhada (PK e FK) nas subclasses de PESSOA. A tabela AUDITORIA, criada na Etapa 2, usa BIGINT porque cresce mais rápido que as demais (Seção 13.2).

5.3 Verificação de normalização

A análise da Etapa 1 (1FN, 2FN e 3FN para as dez relações, com todas as dependências funcionais partindo de superchaves, o que também atende à forma normal de Boyce-Codd) continua válida e não foi alterada pelos objetos criados na Etapa 2. A relação AUDITORIA, nova, também está em 3FN: todos os atributos dependem apenas de id_auditoria. Há, porém, duas **redundâncias controladas** que a Etapa 1 não registrava, uma já justificada e outra apenas implícita:

Atributo	Por que é redundante	Por que foi mantido	Como a consistência é mantida
item_venda.preco_congelado	Poderia ser lido de produto.preco_venda	É um fato histórico da venda (RN07), não uma dependência do preço vigente	Gravado no INSERT do item, com o preço lido sob FOR UPDATE; nunca atualizado depois
venda.valor_total	É a soma de quantidade × preco_congelado dos itens	Evita juntar com item_venda nas consultas de período (B4, estudo 1) e na visão de vendas diárias	Calculado e gravado na mesma transação da venda (atualizarTotal); venda concluída não é alterada

A consistência de valor_total foi **verificada novamente** sobre o banco com a carga e a massa do B4 (200.073 vendas): nenhuma venda com total diferente da soma dos itens, nenhuma venda sem itens e nenhuma venda concluída cuja soma dos pagamentos difira do total. A consulta de verificação fica registrada para reuso:

```
SELECT COUNT(*) AS divergentes
```

```
FROM venda v
JOIN (SELECT id_venda, SUM(quantidade * preco_congelado) AS soma
      FROM item_venda GROUP BY id_venda) x ON x.id_venda = v.id_venda
WHERE v.valor_total <> x.soma;          -- resultado obtido: 0
```

6. Registro de revisões da Etapa 1

O quadro lista o que foi encontrado ao comparar o relatório da Etapa 1 com o DDL e com o código da Etapa 2, e o tratamento dado a cada ponto. Os itens de "alinhamento" já estão refletidos nas Seções 2 e 4 deste relatório; os itens marcados como **decisão** mudam o comportamento do sistema e dependem de confirmação da equipe, com o SQL correspondente no Apêndice B.

#	Ponto	Etapa 1 (original)	Situação real	Tratamento
1	pessoa.nome	VARCHAR(150)	VARCHAR(100)	Alinhamento ao DDL
2	status (pessoa, produto, venda, devolução)	VARCHAR(10)	VARCHAR(20)	Alinhamento ao DDL
3	funcionario.login e senha_hash	login VARCHAR(50); senha_hash ausente	login VARCHAR(100); senha_hash VARCHAR(255) NOT NULL	Dicionário completado (RNF02)
4	produto.codigo_barras	VARCHAR(20)	CHAR(13)	Alinhamento; CHAR(13) é adequado ao EAN-13
5	Valores monetários	DECIMAL(10,2)	preços e preco_congelado: DECIMAL(5,2); valor_total e valor_pago: DECIMAL(8,2)	Alinhamento; **decisão**: ampliar preços (B7, Seção 13.1)
6	devolucao	data_troca DATE; motivo VARCHAR(255)	DATETIME; VARCHAR(200)	Alinhamento ao DDL
7	venda.id_cliente	Dicionário: não nulo; modelo lógico: nulo	Nulo (venda sem cadastro)	Dicionário corrigido: opcional
8	RN18 × cliente.data_nascimento	RN18: só nome e CPF obrigatórios	data_nascimento NOT NULL (necessária a RN06 e RN15)	**Decisão**: RN18 reescrita como nome, CPF e data de nascimento
9	RN05 × exclusão de cliente	Cliente com histórico só se inativa	CRUDCliente.deletar existe e fk_venda_cliente é SET NULL: a venda perderia o cliente	**Decisão**: bloquear a exclusão de cliente com vendas (Apêndice B, item 6)
10	Normalização (A5)	"Nenhuma relação desnormalizada"	valor_total é redundância controlada	Registrada na Seção 5.3
11	Objetos criados depois	—	Tabela auditoria, 2 gatilhos, 3 visões, índices idx_produto_nome e idx_pessoa_nome	Incorporados às Seções 4, 11 e 12
12	Índices do B4 × DDL	B4 informa que os índices mantidos foram incorporados ao 01_ddl.sql	O DDL ainda tem idx_venda_data_hora e não tem idx_venda_periodo_cobre nem idx_item_venda_produto_qtd	**Decisão**: sincronizar (Apêndice B, item 1)
13	B4: lacunas no texto	Placeholders [X] ms e [Y] ms nos quadros de resumo	O Quadro 5 do B4 traz 198 ms e 104 ms	Preenchidos na Seção 10 (1,9 vez)

Quadro 4 — Divergências encontradas e tratamento.

Parte II — Etapa 2

Aplicação, acesso a dados, transações, desempenho, segurança, auditoria e projeto físico

7. Aplicação (B1)

A aplicação é um sistema desktop em Java (Swing, projeto NetBeans) com cerca de 6.200 linhas em três pacotes, separados por responsabilidade: `visao` (10 telas, sem SQL), `controle` (classes de domínio, Sessao e Senha) e `entidade` (DAO e classes CRUD, onde está todo o SQL).

Tela	Função	Perfil
InterfaceLogin	Autentica pelo login e verifica o hash PBKDF2; abre a sessão	Todos
InterfaceInicio	Menu principal; habilita os botões conforme o cargo	Todos
InterfaceVenda	Carrinho, cliente opcional, forma de pagamento e finalização da venda em uma transação	Operador e Gerente
InterfaceCliente, InterfaceFuncionario	CRUD das duas especializações de PESSOA (duas tabelas por transação)	Gerente
InterfaceProduto, InterfaceCategoria	CRUD de catálogo, inativação de produto, percentual de reajuste	Gerente
InterfaceConsultaVendas, InterfaceConsultaProdutos	Consultas com junção e agregação, com filtros opcionais	Gerente
InterfaceRelatorio	Resumo diário de vendas, lido da visão vw_resumo_vendas_diarias	Gerente

Quadro 5 — Telas da aplicação. O perfil Operador só tem habilitado o botão de venda (InterfaceInicio).

7.1 A operação central: registrar uma venda

`CRUDVenda.registrarVenda` concentra o requisito RNF03 (a venda é indivisível). Tudo ocorre em uma única conexão, com `setAutoCommit(false)`:

Passo	O que acontece	Regra
1	Valida o carrinho, soma itens duplicados e ordena por id_produto	Evita deadlock (todas as vendas travam na mesma ordem)
2	Se há cliente, lê a idade no banco e exige cliente ativo	RN06
3	INSERT do cabeçalho da venda com NOW() e o funcionário da sessão	RN11
4	Para cada produto: SELECT ... FOR UPDATE; exige status Ativo; confere a classificação etária	RN05, RN06
5	INSERT do item com o preço lido (preco_congelado)	RN07
6	Produto simples: UPDATE protegido (WHERE quantidade_estoque >= ?); kit: baixa dos componentes	RN01, RN08
7	INSERT do pagamento; UPDATE do valor_total; crédito de pontos ao cliente	RN03, RN20
8	COMMIT. Qualquer exceção leva a ROLLBACK; o finally restaura o autocommit e fecha a conexão	RNF03

O método aceita um parâmetro `simularFalha` que lança um erro depois de gravar itens e dar baixa no estoque e antes do pagamento. Ele existe para **demonstrar o ROLLBACK** na apresentação: a contagem de vendas e o estoque permanecem como antes.

A venda também protege contra a corrida entre dois caixas: como o UPDATE de estoque só altera a linha se houver saldo, e o número de linhas afetadas é conferido, duas vendas simultâneas do último exemplar não deixam o saldo negativo; a segunda recebe a mensagem “Estoque insuficiente”. Além disso, o CHECK do banco funciona como última barreira.

8. Camada de acesso a dados (B2)

Todo o código que fala com o MySQL está em `br.com.entidade`, em JDBC puro. A classe `DAO` lê a configuração, abre e fecha a conexão; cada entidade tem uma classe `CRUD*` que herda de DAO. O detalhamento e as evidências estão em `docs/camada_de_acesso.pdf`; aqui ficam os pontos que sustentam o restante do projeto.

Consultas parametrizadas. Todas as instruções usam PreparedStatement, com valores ligados por setString, setInt e similares. Nas consultas com filtro opcional (CRUDConsulta), o texto do SQL cresce apenas com fragmentos fixos escritos no código (por exemplo, AND pg.forma_pagamento = ?), e o valor do filtro é ligado depois.

Menor privilégio já na conexão. A conexão é aberta com o usuário MySQL do perfil de quem fez login (`Sessao.getPerfilBanco()`): gerente, operador, leitura ou auth. Logo após abrir, o DAO grava o funcionário da sessão em `@id_funcionario_app`, valor que os gatilhos de auditoria (B6) usam.

Teste de injeção de SQL	Entrada	Comando enviado	Resultado
Login	`' OR '1'='1' --`	... FROM vw_login WHERE login = ?	Nenhuma linha; a tela exibe “Login ou senha inválidos.”
Cadastro de categoria	`x'); DROP TABLE categoria; --`	INSERT INTO categoria(nome, ...) VALUES (?, ?)	O texto é gravado literalmente; a tabela continua existindo

Quadro 6 — Tentativas de injeção de SQL (evidências com captura de tela em docs/camada_de_acesso.pdf).

Camadas adicionais: o usuário app_auth só lê a visão vw_login; nenhum perfil tem DROP; a URL de conexão não habilita allowMultiQueries; e o CPF é validado por expressão regular antes de chegar ao banco.

Atenção: credenciais no repositório. O relatório B2 afirma que o arquivo real `config.properties` está listado no .gitignore e que só o arquivo de exemplo é versionado. Na verificação do repositório, porém, **não há .gitignore** e o arquivo `ToyStoreySystem/config.properties` está versionado, com as senhas dos quatro perfis; as mesmas senhas aparecem em `sql/05_acesso.sql`. Como são credenciais de laboratório, o risco real é baixo, mas a afirmação do B2 precisa ser corrigida no repositório (Apêndice B, item 5) antes da apresentação, e as senhas devem ser trocadas se algum dia forem reaproveitadas.

9. Transações e controle de concorrência (B3)

A operação documentada no B3 é o cadastro de cliente (`CRUDCliente.inserir`), que grava em `pessoa` e em `cliente`. O INSERT em pessoa precisa vir primeiro, porque gera o id usado como PK/FK na subclasse; se o segundo comando falhar, o primeiro não pode permanecer. O ciclo é: `setAutoCommit(false)`, os dois INSERTs, `commit()`; em qualquer exceção, `rollback()` e propagação do erro; no `finally`, restauração do autocommit e fechamento da conexão.

Teste	Cenário	Resultado observado
1 — COMMIT	Cadastro do cliente “denji”, CPF 69696969696	Contagem de clientes de 100.051 para 100.052; registro com id_pessoa 200002
2 — ROLLBACK	Novo cadastro reutilizando o mesmo CPF (viola uq_cliente_cpf, erro 1062)	O INSERT em pessoa é desfeito; contagem permanece 100.052; a tela mostra “Já existe um cliente com esse CPF.”
3 — Concorrência	Duas sessões no Workbench: a A altera a linha sem confirmar; a B tenta alterar a mesma linha	A sessão B fica em LOCK WAIT (19 s no print) até a A decidir; sem confirmação, o MySQL devolveria o erro 1205 após innodb_lock_wait_timeout

Quadro 7 — Testes do B3 (docs/Transacoes_e_Concorrencia.pdf).

O nível de isolamento é o padrão do InnoDB, REPEATABLE READ. O bloqueio de linha do UPDATE (exclusivo, mantido até COMMIT ou ROLLBACK) serializa alterações simultâneas na mesma linha, e nenhuma atualização se perde. A venda (Seção 7.1) aplica a mesma ideia com SELECT ... FOR UPDATE ordenado por produto. Erros 1062 (duplicidade), 1213 (deadlock) e 1205 (tempo de

espera) devem ser traduzidos para mensagens compreensíveis; o 1062 já é tratado na tela de cliente.

10. Otimização de consultas (B4)

O estudo completo está em `docs/b4_relatorio.pdf`. Como a carga da Etapa 1 tem poucas dezenas de linhas por tabela, pequena demais para o otimizador preferir um índice, o script `04_volume_b4.sql` gera uma massa sintética determinística (sem dados pessoais reais). O volume foi **reproduzido** na elaboração deste relatório e confere com o Quadro 2 do B4: pessoa 100.090, cliente 100.050, produto 60, venda 200.073, item_venda 500.173, pagamento 192.093 e devolução 4.045 linhas.

Método. EXPLAIN e EXPLAIN ANALYZE no MySQL 8.0 (Windows 11, Ryzen 5 5500, 16 GB, innodb_buffer_pool_size de 128 MB); cinco execuções por medida, descartada a primeira (cache frio) e registrada a mediana das outras quatro; IGNORE INDEX para simular a ausência do índice. Os tempos abaixo são os medidos pela equipe.

Estudo	Situação	type	Índice usado	Linhas estimadas	Tempo	Ganho
1. Vendas por funcionário (trimestre)	Sem índice	ALL	—	199.660	195 ms	
	idx_venda_periodo_cobre (data_hora, status, id_funcionario, valor_total)	range	sim (cobertura)	35.272	37,9 ms	5,1x
2. Cliente por nome	LIKE 'Ana Silva%' sem índice	ALL	—	98.736	21,9 ms	
	LIKE 'Ana Silva%' com idx_pessoa_nome	range	sim	251	0,572 ms	38x
	LIKE '%Silva%' sem índice	ALL	—	98.736	52,8 ms	
	LIKE '%Silva%' com índice	index	varredura completa	98.736	73,8 ms	**piora**
3. Ranking de produtos	Geral, sem índice	ALL	—	488.122	198 ms	
	Geral, com idx_item_venda_produto_qtd (id_produto, quantidade)	index	sim (cobertura)	488.122	104 ms	1,9x
	Período largo (~26% das vendas), sem índice em venda	ALL	—	199.660	517 ms	
	Período largo, com idx_venda_periodo_cobre	range	sim	99.830	473 ms	9%
	Uma semana (~0,7%), sem índice em venda	ALL	—	199.660	163 ms	
	Uma semana, com idx_venda_periodo_cobre	range	sim	1.401	10,5 ms	15,5x

Quadro 8 — Resultados do B4 consolidados. Os dois índices mantidos estão descritos no B4; a linha "Geral" completa os valores que o B4 deixou como [X] e [Y].

10.1 Discussão

- **Ordem das colunas.** No estudo 1, data_hora vem primeiro por ser o filtro de faixa e seletivo (um trimestre é cerca de 9% do período); status vem depois porque tem só dois valores e ~96% das vendas são "Concluída", logo filtrar por ele primeiro não reduziria nada; id_funcionario e valor_total entram só para cobertura (Using index). No estudo 3, id_produto vem primeiro por ser a coluna do GROUP BY.
- **Índice redundante removido.** idx_venda_data_hora passou a ser prefixo do índice de cobertura e deve ser eliminado, reduzindo espaço e custo dos INSERTs; o novo índice de item_venda também atende à FK, e o MySQL descarta o índice automático dela.

- **Resultado negativo (curinga no início).** Com LIKE '%Silva%' o índice foi percorrido inteiro (type = index, 98.736 linhas) e o tempo piorou de 52,8 para 73,8 ms. A causa é a ordenação do índice pelo começo do texto, uma limitação do tipo de índice e não um erro de criação. A alternativa seria FULLTEXT (MATCH ... AGAINST) ou exigir busca por prefixo na interface.
- **Ganho pequeno por baixa seletividade.** No ranking de período largo o índice foi usado, mas o ganho foi de 9%: o filtro mantém uma parcela grande das vendas (99.830 linhas estimadas) e o custo é dominado pela junção e pela tabela temporária. A mesma consulta e o mesmo índice rendem 15,5x na janela de uma semana. Um índice só compensa quando o filtro descarta a maior parte das linhas.
- **Custo.** Índices largos encarecem as escritas; isso é aceitável em venda, que quase não muda depois de concluída. Nenhum índice foi criado sem uma consulta que o justificasse.

11. Segurança e controle de acesso (B5)

11.1 Perfis e princípio do menor privilégio

O script `05_acesso.sql` cria quatro usuários MySQL. Cada um recebe só o necessário; o script começa com REVOKE ALL PRIVILEGES, GRANT OPTION, de modo que reexecutá-lo sempre parte de um estado limpo.

Usuário	Para quem	Privilégios concedidos	Decisão de projeto
app_gerente	Funcionário com cargo Gerente	SELECT, INSERT e UPDATE nas tabelas de negócio; SELECT e INSERT em item_venda e pagamento; SELECT em auditoria; DELETE em pessoa, cliente, funcionario, produto, categoria e kit_composicao	Itens e pagamentos não têm UPDATE (são imutáveis depois de gravados); venda não tem DELETE (cancela-se); a auditoria só pode ser lida
app_operador	Funcionário com cargo Operador	Visão vw_cliente_atendimento; SELECT em colunas de pessoa (id_pessoa, status) e cliente (id_pessoa, data_nascimento, pontos_fidelidade); UPDATE só de pontos_fidelidade; produto: SELECT e UPDATE só de quantidade_estoque; venda: SELECT, INSERT e UPDATE só de valor_total; item_venda e pagamento: SELECT e INSERT; categoria e kit_composicao: SELECT	Privilégios por coluna: o operador não vê CPF, não altera preço nem status, e não cancela venda
app_leitura	Relatórios	SELECT apenas em vw_resumo_vendas_diarias	Acesso a dados agregados, sem linhas de venda
app_auth	Tela de login	SELECT apenas em vw_login	A autenticação não enxerga mais nada do banco

Quadro 9 — Perfis de acesso. O funcionário do perfil Gerente ou Operador é mapeado ao usuário MySQL por Sessao.getPerfilBanco().

11.2 Visões para restringir colunas e linhas

Visão	O que restringe	Usada por
vw_cliente_atendimento	Colunas: expõe nome, pontos e o CPF **mascarado** (***.***.015-**), nunca o CPF completo. Linhas: só clientes ativos	app_operador
vw_login	Colunas: apenas id, login, senha_hash, cargo e nome. Linhas: só funcionários ativos, então um funcionário inativado perde o acesso	app_auth
vw_resumo_vendas_diarias	Entrega agregados por dia (vendas, itens, faturamento, ticket médio), apenas de vendas concluídas	app_leitura

11.3 Armazenamento de senhas

As senhas dos funcionários nunca são gravadas em texto claro (RNF02). A classe Senha usa **PBKDF2 com HMAC-SHA256**, 120.000 iterações, **sal aleatório de 16 bytes** (SecureRandom) gerado por senha, e chave derivada de 256 bits; o valor gravado tem o formato `pbkdf2$iterações$sal$hash`, o que permite aumentar as iterações no futuro sem invalidar os hashes antigos. A verificação lê as iterações do próprio registro e compara os bytes em tempo constante (`MessageDigest.isEqual`), evitando vazamento por tempo de resposta. O sal individual impede que duas senhas iguais produzam o mesmo hash e inutiliza tabelas pré-calculadas.

11.4 Dados pessoais do projeto e cuidados adotados

A Lei Geral de Proteção de Dados (Lei nº 13.709/2018) define dado pessoal como informação relacionada a pessoa natural identificada ou identificável. O quadro registra quais dados do banco se enquadram, como estão protegidos e o que ainda é recomendado.

Dado	Onde	Por que é pessoal	Cuidado adotado	Recomendação
Nome	pessoa.nome	Identifica diretamente	Operador só lê a visão e id/status; consultas por perfil	—

Dado	Onde	Por que é pessoal	Cuidado adotado	Recomendação
CPF	cliente.cpf	Identificador único nacional	UNIQUE; validação por regex; máscara na visão do operador; coluna sem permissão ao operador	Não replicar o CPF em auditoria nem logs; criptografar backups
Data de nascimento	cliente.data_nascimento	Identificável; revela a idade, e numa loja de brinquedos há clientes menores	Operador lê só para calcular a classificação etária (RN06)	Dados de crianças e adolescentes têm regime próprio (art. 14): coletar só o necessário e, para menores, o consentimento de um responsável
Login e hash de senha	funcionario	Credencial de trabalhador	Hash PBKDF2 com sal; visão vw_login só para app_auth	Nunca incluir em relatórios
Pontos de fidelidade e histórico de compras	cliente, venda.id_cliente	Perfil de consumo associado a uma pessoa	Acesso por perfil; histórico preservado por RN05	Definir prazo de retenção; anonimizar em vez de excluir quando o titular pedir eliminação (art. 18)
Autor das alterações	auditoria.id_funcionario	Trabalhador identificável	Auditoria legível só pelo gerente	Informar a finalidade aos funcionários

Quadro 10 — Inventário de dados pessoais. Nenhum dado pessoal sensível (art. 5º, II) é coletado.

Também fazem parte dos cuidados: credenciais do banco fora do código (Seção 8), consultas parametrizadas, ausência de DROP e de allowMultiQueries, auditoria de preços e estoque, e a massa do B4, gerada por fórmulas e sem dados reais. Duas recomendações adicionais: habilitar TLS na conexão JDBC quando o banco deixar de ser local, e tratar o conflito entre guardar o histórico (RN05) e o direito de eliminação do titular por anonimização.

11.5 Verificação das permissões

Para esta entrega, os perfis foram **exercitados** em um banco de referência (MariaDB 10.11 com o 05_acesso.sql executado sem alterações). Recomenda-se repetir os mesmos comandos no MySQL 8.0 da equipe e guardar as capturas de tela para a apresentação.

Conexão como	Comando	Resultado
app_operador	SELECT cpf FROM cliente	Negado: erro 1143 (coluna cpf)
app_operador	SELECT * FROM vw_cliente_atendimento	Permitido; CPF mascarado
app_operador	SELECT senha_hash FROM funcionario	Negado: erro 1142
app_operador	UPDATE produto SET preco_venda = 1 ...	Negado: erro 1143 (coluna preco_venda)
app_operador	DELETE FROM venda ...	Negado: erro 1142
app_operador	SELECT * FROM auditoria	Negado: erro 1142
app_operador	UPDATE produto SET quantidade_estoque = ...	Permitido, e o gatilho grava a linha de auditoria (a gravação ocorre com os direitos do gatilho, não do operador)
app_leitura	SELECT * FROM venda	Negado: erro 1142; a visão diária é legível
app_auth	SELECT * FROM pessoa	Negado: erro 1142; vw_login é legível
app_gerente	DELETE FROM auditoria	Negado: erro 1142; a trilha não é apagável pelo gerente

Quadro 11 — Verificação das permissões (execução de referência feita na elaboração deste relatório).

Ponto de atenção. app_gerente recebe DELETE em pessoa, cliente, funcionario, produto, categoria e kit_composicao. As FKs protegem produto e categoria em uso (RESTRICT), mas não protegem o histórico de um cliente (SET NULL), o que conflita com RN05; ver Seção 6, item 9, e Apêndice B, item 6.

12. Gatilhos e auditoria (B6)

12.1 Estrutura da trilha

A tabela `auditoria` registra, a cada alteração relevante, o quê (tabela, id do registro e campo), o valor anterior e o posterior, **quem** (id do funcionário da aplicação e usuário do banco) e **quando** (DATETIME com valor padrão CURRENT_TIMESTAMP). Uma linha por campo alterado deixa a consulta simples e permite auditar campos diferentes do mesmo registro de forma independente. Há índices por (tabela, id_registro), para consultar o histórico de um item, e por data_hora, para consultar um período. Nenhum perfil da aplicação tem INSERT, UPDATE ou DELETE em auditoria; ela só é gravada pelos gatilhos.

12.2 Os dois gatilhos e a justificativa pelo domínio

Gatilho	Dispara em	Registra	Por que auditar no domínio da loja
trg_produto_auditar_preco	AFTER UPDATE em produto, quando preco_venda ou preco_custo mudam	Uma linha por campo alterado, com valores anterior e posterior	Preço de venda e custo determinam faturamento e margem; o reajuste em lote (RN19) mexe em muitos produtos de uma vez; a trava de preço mínimo (RN14) depende do custo. Uma alteração indevida de preço é a fraude mais direta numa loja
trg_produto_auditar_estoque	AFTER UPDATE em produto, quando quantidade_estoque muda	Uma linha com o saldo anterior e o posterior	O estoque muda a cada venda, mas também por perdas, avarias e ajustes manuais (RN01, RN17); a trilha permite reconstituir a movimentação e achar divergência de inventário

O teste de igualdade usa o operador <=> (igualdade segura para NULL), e o gatilho só grava quando o valor realmente mudou: um UPDATE que não altera nada não gera linha. Fica de fora, por decisão: a alteração de cadastro de cliente, que não afeta o dinheiro da loja e obrigaria a guardar CPF e data de nascimento antigos na auditoria, ampliando a exposição de dado pessoal (Seção 11.4).

12.3 Evidência de funcionamento

Verificação feita em um banco de referência com a carga da Etapa 1 (a variável de sessão `@id_funcionario_app` simula o que o DAO faz ao abrir a conexão):

Ação	Linhas em auditoria
UPDATE produto SET preco_venda = preco_venda + 1 (produto 1)	1 linha: preco_venda 25.26 para 26.26, id_funcionario 1
UPDATE produto SET quantidade_estoque = quantidade_estoque - 2 (produto 1)	1 linha: quantidade_estoque 179 para 177, id_funcionario 1
UPDATE produto SET nome = nome (produto 2; nada muda)	Nenhuma linha
Baixa de 1 unidade do produto 3 pelo usuário app_operador, funcionário 5	1 linha: 51 para 50, id_funcionario 5

Quadro 12 — Comportamento dos gatilhos (execução de referência).

12.4 Defeito encontrado: o usuário do banco gravado é sempre o definidor do gatilho

Na última linha acima, a coluna `usuario_banco` foi gravada como **root@localhost**, embora a alteração tenha sido feita por **app_operador**. Isso ocorre porque os gatilhos usam `CURRENT_USER()`, que dentro de uma rotina com SQL SECURITY DEFINER devolve a conta do *definidor* (quem criou o gatilho), e não a de quem executa a alteração. Foi confirmado com um gatilho de teste, que gravou os dois valores lado a lado: CURRENT_USER() = root@localhost e USER() = app_operador@localhost. O campo, como está, não identifica ninguém. A correção é trocar por `USER()`:

```
-- nos dois gatilhos, em cada INSERT INTO auditoria (...):
```

```
VALUES ('produto', NEW.id_produto, 'preco_venda', OLD.preco_venda,
        NEW.preco_venda, @id_funcionario_app, USER());   -- era CURRENT_USER()
```

O autor lógico (`id_funcionario`) continua correto, pois vem da aplicação; só o campo de usuário do banco precisa da correção. Um limite que deve constar na apresentação: se alguém alterar o produto diretamente no banco, sem passar pelo DAO, `@id_funcionario_app` estará nulo e só o usuário do banco identificará o autor, por isso a correção importa.

13. Projeto físico e armazenamento (B7)

Esta seção responde aos quatro itens do B7. Quando aparece medição, ela vem de um banco de referência carregado com o 01_ddl.sql e a massa do B4 (volume idêntico ao do Quadro 2 do B4), com os índices finais do B4 aplicados. O mecanismo usado foi MariaDB 10.11 (InnoDB), próximo do MySQL 8.0 usado pela equipe, mas não idêntico: os tamanhos por linha são uma boa aproximação, e vale repetir a medição no ambiente da equipe antes de citar números exatos na apresentação. As decisões, por sua vez, são propostas para a equipe confirmar.

13.1 Revisão dos tipos de dados

Campo(s)	Tipo atual	Avaliação	Decisão proposta
produto.preco_custo, produto.preco_venda, item_venda.preco_congelado	DECIMAL(5,2): máximo 999,99	A carga chega a 319,31, mas um brinquedo caro (console, bicicleta) ou um kit estoura o limite e o banco rejeitaria o cadastro. valor_total já é DECIMAL(8,2), então um item poderia ser maior que a própria venda	**Mudar** para DECIMAL(8,2): custo de 1 byte por valor, sem efeito perceptível no tamanho
produto.codigo_barras	CHAR(13)	Código EAN-13 tem tamanho fixo; CHAR evita o byte de tamanho do VARCHAR e acelera comparação	Manter
cliente.cpf	CHAR(11)	Tamanho fixo, só dígitos; a máscara é feita na visão	Manter
status, cargo, tipo, forma_pagamento, tipo_destino	VARCHAR(20) com CHECK	ENUM ou TINYINT com tabela de domínio economizaria cerca de 9 bytes por linha (~2 MB em venda no volume do B4), mas tornaria o domínio menos legível e menos portável; o CHECK já garante os valores	Manter; só reavaliar se o volume crescer muito
venda.data_hora, devolucao.data_troca	DATETIME	5 bytes, sem conversão de fuso, faixa ampla; segundos bastam	Manter
pessoa.nome	VARCHAR(100)	Suficiente para nome completo; o utf8mb4 reserva até 4 bytes por caractere apenas na memória, não em disco	Manter
id_* (chaves)	INT; BIGINT em auditoria.id_auditoria	INT suporta 2,1 bilhões de linhas, muito acima da projeção; auditoria cresce mais e já usa BIGINT	Manter
auditoria.valor_anterior, valor_posterior	VARCHAR(100)	Texto genérico guarda preço e quantidade na mesma coluna, ao custo de perder o tipo numérico	Manter; é o preço da generalidade da trilha

Quadro 13 — Revisão de tipos. Somente os preços têm mudança proposta.

13.2 Estimativa de crescimento em dois anos

Premissas. A massa do B4 tem 200.073 vendas em cerca de 33 meses (janeiro de 2024 a setembro de 2026), ou cerca de 6.000 vendas por mês; mantendo esse ritmo, em 24 meses são ~146.000 vendas. Cada venda tem em média 2,5 itens, 0,96 pagamento e 2% de devoluções, e o gatilho de estoque grava uma linha de auditoria por item vendido. Os bytes por linha foram **medidos** (dados + índices, com os índices finais do B4):

Tabela	Bytes por linha (medido)	Linhas em 24 meses	Crescimento estimado
venda	142	146.000	20,7 MB
item_venda	68	365.000	24,8 MB
pagamento	66	140.200	9,3 MB
devolucao	106	2.920	0,3 MB
pessoa + cliente (1 cliente para 2 vendas, como na massa)	117 + 74	73.000	14,0 MB
auditoria (1 linha por item vendido)	222	365.000	**81,0 MB**
Total			**~150 MB**

Quadro 14 — Projeção de crescimento. O banco da massa do B4 ocupa cerca de 90 MB; em dois anos ficaria em torno de 240 MB.

- **A tabela que mais cresce não é a de vendas, é a de auditoria:** o gatilho de estoque a transforma em 54% do crescimento projetado, com linhas largas (222 bytes) por causa dos dois VARCHAR(100) e dos dois índices.
- **Sazonalidade.** Uma loja de brinquedos concentra vendas em dezembro e em datas como o Dia das Crianças; o ritmo médio subestima os picos, mas o total anual muda pouco e a conclusão não se altera.
- **Memória.** O conjunto de trabalho (~240 MB em dois anos) já excede o innodb_buffer_pool_size de 128 MB usado nos testes do B4. Recomenda-se 512 MB para que dados e índices quentes caibam na memória.

13.3 Plano de backup

Aspecto	Proposta	Justificativa
O que copiar	O banco loja_toystorey inteiro (dados, visões, gatilhos, rotinas e eventos) e, em arquivo à parte, as contas e privilégios (SHOW GRANTS dos quatro usuários)	O dump lógico não inclui os usuários; sem eles a restauração subiria um banco que a aplicação não consegue abrir
Ferramenta	mysqldump com --single-transaction, --routines, --triggers, --events e --no-tablespaces, por um usuário dedicado (SELECT, SHOW VIEW, TRIGGER, EVENT)	Em InnoDB, --single-transaction gera cópia consistente sem bloquear a venda; um usuário próprio segue o menor privilégio
Frequência	Completo uma vez por dia, às 02:00 (fora do horário da loja, conforme RNF04), agendado no Agendador de Tarefas do Windows; binlog ativado para recuperação até um instante	O RPO fica em até 24 h só com o dump, e em minutos com o binlog
Retenção	7 diários, 4 semanais e 12 mensais, com cópia em segundo local (outro disco ou nuvem)	Cobre erro percebido tarde (mensal), falha do disco (segundo local) e uso cotidiano (diário). Com ~15 MB por dump compactado em dois anos, as 23 cópias somam ~350 MB
Proteção	Dump compactado e **criptografado**, com acesso restrito	O dump contém CPF e data de nascimento (Seção 11.4)
Teste de restauração	Mensal: restaurar o dump mais recente em um esquema temporário, comparar contagens e CHECKSUM TABLE das tabelas, conferir gatilhos e visões e rodar as 15 consultas de 03_consultas.sql; registrar data e tempo	Backup que nunca foi restaurado não é backup. O teste também mede o RTO real

Quadro 15 — Plano de backup (proposta).

Ensaio de restauração realizado. Para validar o procedimento, o banco de referência (massa do B4 mais 100.000 linhas sintéticas de auditoria) foi copiado com mysqldump (`--single-transaction --routines --triggers --events`) e restaurado em outro esquema. Resultado: dump de 46 MB (6,8 MB compactado, relação de ~7 para 1), gerado em menos de 1 s e restaurado em ~9,5 s; contagens idênticas em venda (200.073), item_venda (500.173), pagamento (192.093) e auditoria (100.003); CHECKSUM TABLE igual na origem e no restaurado para venda e item_venda; 2 gatilhos e 3 visões restaurados. O RTO de projeto (30 minutos) é dominado pela preparação do servidor, não pela restauração. Os tempos foram medidos no ambiente de referência e dependem do hardware.

13.4 Particionamento e arquivamento histórico

Conclusão: nenhuma tabela justifica particionamento agora; a auditoria justifica arquivamento.

- **Por que não particionar venda, item_venda ou pagamento.** Mesmo em dois anos, a maior tabela de movimento (item_venda) terá cerca de 0,9 milhão de linhas (~59 MB); índices bem escolhidos (B4) resolvem as consultas de período, como mostrou o ganho de 15,5x na janela de uma semana. Além disso, o InnoDB do MySQL não aceita chave estrangeira em tabela

particionada, nem referência a uma: venda é referenciada por item_venda, pagamento e devolucao, e particioná-la exigiria abrir mão da integridade referencial que sustenta RN05, RN09 e RN11. A chave de partição ainda teria de compor toda chave única, inclusive a PK.

- **Por que arquivar a auditoria.** É a tabela de maior crescimento (81 MB em 24 meses), é só de leitura depois de gravada, e a consulta típica olha o passado recente. Proposta: manter 12 meses em `auditoria` e mover o restante, mensalmente e em lotes, para `auditoria_historico` (mesma estrutura, sem gatilhos), por um evento agendado (Event Scheduler). O histórico mais antigo pode ser exportado e compactado. Como a tabela tem FK para funcionario, o arquivamento por INSERT ... SELECT seguido de DELETE em lotes funciona sem tocar em particionamento.
- **Gatilho de revisão.** Reavaliar se uma única tabela passar de alguns GB ou se a mesma consulta de período, mesmo com índice de cobertura, passar a ser lenta; nesse cenário, o caminho é arquivar vendas antigas e não particionar, enquanto houver FKs.

Parte III — Fechamento

Avaliação do atendimento, pendências, lições aprendidas e conclusão

14. Avaliação do atendimento aos requisitos e pendências

14.1 Requisitos funcionais e não funcionais

Requisito	Descrição	Situação	Evidência / observação
RF01	Frente de caixa (PDV)	Parcial	Tela de venda com carrinho, cliente opcional, pagamento e transação (InterfaceVenda, CRUDVenda). Leitor de código de barras, troco e atalhos de teclado devem ser confirmados na tela antes da apresentação
RF02	Controle de acessos por perfil	Implementado	Login com hash; menu por cargo; usuário MySQL por perfil; permissões por coluna (Seção 11)
RF03	Trocas e devoluções	Pendente	Tabela devolucao e consulta de verificação prontas; falta a tela e a rotina (RN09, RN10, RN17)
RF04	Alerta de reposição	Parcial	estoque_minimo e a consulta 13 (itens abaixo do mínimo); sem alerta na interface
RF05	Fechamento de caixa	Parcial	Consulta por funcionário e forma de pagamento (CRUDConsulta); sem rotina de encerramento de turno
RNF01	Item no carrinho em até 300 ms	Pendente	Não medido. O código de barras tem índice UNIQUE (acesso por ref); falta registrar o tempo com EXPLAIN ANALYZE
RNF02	Senhas com hash	Atendido	PBKDF2-HMAC-SHA256, sal de 16 bytes, 120.000 iterações (Seção 11.3)
RNF03	Venda indivisível	Atendido	Transação com COMMIT e ROLLBACK (Seções 7.1 e 9)
RNF04	Backup diário automático	Parcial	Plano e ensaio de restauração feitos (Seção 13.3); falta criar a tarefa agendada

Quadro 16 — Atendimento aos requisitos.

14.2 Pendências priorizadas

Os cinco primeiros itens são correções pequenas que tornam o repositório coerente com os relatórios; convém fazê-las antes da apresentação, porque o professor pode abrir o DDL ou o config.properties durante a arguição.

#	Pendência	Esforço	Referência
1	Sincronizar o 01_ddl.sql com o B4 (dois índices novos, remover idx_venda_data_hora)	Baixo	Apêndice B, item 1
2	Trocar CURRENT_USER() por USER() nos dois gatilhos	Baixo	Seção 12.4; Apêndice B, item 2
3	Corrigir o crédito de pontos para 1 ponto a cada R$ 100,00 (RN20)	Baixo	Apêndice B, item 3
4	Criar .gitignore e retirar config.properties (e build/) do versionamento; corrigir o texto do B2	Baixo	Apêndice B, item 5
5	Impedir a exclusão de cliente com vendas (RN05) e tratar o erro 1451 na tela	Baixo a médio	Apêndice B, item 6
6	Ampliar preços para DECIMAL(8,2)	Baixo	Seção 13.1; Apêndice B, item 4
7	Criar a tarefa agendada de backup e o usuário de backup	Médio	Seção 13.3
8	Repetir as medições de B5 e B7 no MySQL 8.0 e guardar as capturas de tela	Baixo	Seções 11.5 e 13

#	Pendência	Esforço	Referência
9	Decidir o escopo de RF03 (devolução), RN04 e RN14 (desconto) e RN13 (cancelamento): implementar ou declarar como trabalho futuro	Alto	Quadro 1
10	Medir o RNF01 e confirmar os detalhes do RF01 na tela de venda	Baixo	Quadro 16

15. Conclusão e lições aprendidas

O projeto entrega um banco relacional normalizado (3FN), com restrições nomeadas, 15 consultas de verificação, uma aplicação em camadas que conversa com o SGBD por consultas parametrizadas e por perfis de menor privilégio, operações transacionais com COMMIT e ROLLBACK demonstráveis, três estudos de desempenho com um resultado negativo bem analisado, auditoria de preço e estoque, e um projeto físico com projeção de crescimento, plano de backup testado e uma decisão fundamentada sobre particionamento.

- **Índice não é sinônimo de rapidez.** O mesmo índice rendeu 38x, 15,5x, 5,1x, 1,9x, 9% e foi pior que a leitura completa, conforme a seletividade do filtro e a posição do curinga. Planos de execução precisam ser lidos antes e depois.
- **Documentação só vale se for conferida contra o código.** Ao consolidar, apareceram divergências entre dicionário e DDL, uma regra (RN20) implementada com outro valor, um .gitignore que não existe e índices descritos mas não aplicados. Todas são corrigíveis em minutos, e é melhor que a equipe as encontre antes da banca.
- **Segurança em camadas.** Parametrização, menor privilégio por coluna, visões com máscara, hash com sal e credenciais fora do código se complementam; nenhuma, sozinha, basta.
- **Auditoria também tem custo.** O gatilho de estoque faz da auditoria a tabela que mais cresce, o que obriga a pensar em retenção e arquivamento desde o início.
- **Armadilhas de SGBD aparecem no teste, não na leitura:** CURRENT_USER() dentro de gatilho devolve o definidor. Só a execução com um usuário diferente revelou o defeito.

16. Uso de inteligência artificial

O uso de assistentes de IA foi declarado pela equipe no README conforme o enunciado: apoio na escrita de scripts SQL, esclarecimento de conceitos, revisão de clareza dos documentos e unificação dos relatórios da Etapa 1. Para este relatório consolidado, a IA foi usada para ler o repositório, reexecutar os scripts em um banco de referência, comparar documentos com código e redigir o texto. Todo o conteúdo precisa ser verificado e validado pela equipe, que se responsabiliza integralmente pelo material submetido; os pontos marcados como proposta ou decisão (Quadros 4, 13 e 15) dependem dessa confirmação.

Referências

- MOURA JÚNIOR, S. N. *Enunciado do projeto final de Laboratório de Banco de Dados* (GPE17M40083). Universidade Católica de Brasília, 2026.
- ORACLE. *MySQL 8.0 Reference Manual*: EXPLAIN; mysqldump; Stored Object Access Control; Restrictions and Limitations on Partitioning.
- BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais).
- MOORE, K.; KALISKI, B. *PKCS #5: Password-Based Cryptography Specification Version 2.0* (RFC 8018). IETF, 2017.
- Documentos da equipe em docs/: relatorio-etapa1, b4_relatorio, camada_de_acesso, Transacoes_e_Concorrencia.

Apêndice A — Roteiro da apresentação (15 minutos)

Cronograma, demonstração ao vivo, preparação e perguntas prováveis

A apresentação tem 15 minutos, com demonstração da aplicação em execução (B8). O cronograma deixa 4 minutos e meio para a demonstração, que é o que mais pesa, e fecha dentro do tempo; a divisão entre os três integrantes fica a critério da equipe (sugestão: um integrante por bloco temático, o mesmo que opera a demonstração).

Tempo	Bloco	Conteúdo	Apoio visual
0:00–1:00	Abertura	O que é a Toy Storey, o problema (ciclo completo da venda) e o roteiro	Slide de título + 1 frase por etapa
1:00–3:00	Domínio e modelo	Entidades-chave, especialização de PESSOA, kits como autorrelacionamento, 20 regras de negócio e onde cada uma é garantida	MER (Figura 1) e Quadro 1 resumido
3:00–4:30	Etapa 1 revisada	3FN; as duas redundâncias controladas (preco_congelado e valor_total); o que a revisão encontrou	Quadro 4 (3 a 4 linhas) + consulta de verificação
4:30–6:00	Arquitetura e acesso	Três pacotes; PreparedStatement; um usuário MySQL por perfil; credenciais fora do código	Diagrama de camadas + Quadro 9
6:00–10:30	**Demonstração ao vivo**	Ver o roteiro detalhado abaixo	Aplicação + MySQL Workbench
10:30–12:30	Desempenho (B4)	EXPLAIN antes e depois; o caso do curinga que piora; a mesma consulta com 15,5x e 9% conforme a seletividade	Quadro 8 + um EXPLAIN real em tela
12:30–13:45	Projeto físico (B7)	A auditoria é a tabela que mais cresce; plano de backup e ensaio de restauração; por que não particionar	Quadros 14 e 15
13:45–14:30	Resultados e limites	O que está pronto, o que é pendência declarada, lições	Quadro 16 (resumo)
14:30–15:00	Encerramento	Síntese e abertura para perguntas	Slide final

A.1 Demonstração ao vivo (4 min 30 s)

Min	Ação na tela	O que mostrar / dizer
6:00	Login como **Operador**	Apenas o botão de venda está habilitado (RF02). A conexão usa app_operador
6:30	Venda para um cliente **menor que a classificação** do brinquedo	A venda é recusada com mensagem clara (RN06); nada é gravado
7:15	Venda válida de um **kit** a um cliente cadastrado	Preço congelado no item; baixa nos componentes (RN08); pontos creditados ao cliente; total e pagamento coerentes
8:15	Repetir uma venda com a **falha simulada**	ROLLBACK: o SELECT COUNT(*) em venda e o estoque não mudam (RNF03)
9:00	No Workbench, conectado como **app_operador**	SELECT cpf FROM cliente é negado (1143); vw_cliente_atendimento mostra o CPF mascarado
9:30	Ainda como operador: ajustar o estoque de um produto; depois ver a auditoria como root ou gerente	O gatilho gravou valor anterior, posterior e o funcionário. Mostrar **usuario_banco** já com a correção do Apêndice B, item 2 (app_operador, não root)
10:00	Login como **Gerente**: relatório diário	Todos os botões habilitados; o relatório lê a visão agregada

A.2 Checklist de preparação (véspera e antes de entrar)

- Banco recriado do zero: 01_ddl.sql, 02_carga.sql, 05_acesso.sql; para o B4, também o 04_volume_b4.sql.
- **Executar o GeradorSenhas** e anotar a senha de demonstração (a carga traz hashes fictícios, e sem isso nenhum login funciona).
- Correções 1 a 5 do Quadro de pendências aplicadas no banco e no repositório.
- config.properties local preenchido; projeto compilado e testado em outra máquina, se possível a que será usada na apresentação.
- Separar com antecedência: um cliente menor de idade, um produto com classificação maior que a idade dele, um kit com estoque dos componentes, um produto com estoque baixo.
- Workbench com duas abas já conectadas (app_operador e root) e os comandos prontos; capturas de tela de reserva de cada passo da demonstração.
- Ensaiar com cronômetro. Se a demonstração atrasar, cortar o passo do gerente (10:00) e não o do ROLLBACK.

A.3 Perguntas prováveis e respostas curtas

Pergunta	Resposta
Por que PBKDF2 e não um SHA-256 simples?	Hash rápido facilita ataque de força bruta. PBKDF2 repete a derivação 120.000 vezes e o sal individual impede tabelas pré-calculadas; o formato guarda as iterações, então dá para aumentá-las depois
Como a venda evita estoque negativo com dois caixas?	SELECT ... FOR UPDATE em ordem de id_produto e UPDATE condicionado a quantidade_estoque >= ?; se nenhuma linha muda, a venda falha. O CHECK do banco é a última barreira
Por que o índice do LIKE '%Silva%' piorou?	O índice é ordenado pelo início do texto; sem prefixo não há ponto de partida e o MySQL percorre o índice inteiro, o que custa o mesmo que a tabela e ainda lê a estrutura do índice. A saída é FULLTEXT ou exigir prefixo
Por que a coluna status vem depois de data_hora no índice?	data_hora filtra uma faixa seletiva; status tem 2 valores e 96% das vendas são “Concluída”, então colocá-la primeiro quase não reduziria linhas
Por que não particionaram a tabela de vendas?	O volume não justifica e o InnoDB não aceita FK em tabela particionada; perderíamos RN05, RN09 e RN11. A auditoria, esta sim, é arquivada
Por que CURRENT_USER() não serve no gatilho?	Em rotina com SQL SECURITY DEFINER ele devolve quem criou o gatilho; USER() devolve quem está conectado. Foi o que o teste com app_operador revelou
Como vocês testam o backup?	Restauração mensal em esquema temporário, comparando contagens e CHECKSUM TABLE, conferindo gatilhos e visões e medindo o tempo; no ensaio, 46 MB restauraram em ~9,5 s com checksums idênticos
O que mudaria com mais tempo?	Devolução (RF03) e desconto (RN04, RN14), com o tratamento de RN10 e RN13; automatizar o backup; FULLTEXT para busca de nome

Apêndice B — Correções propostas ao repositório

Alterações pequenas que alinham o código aos relatórios; marcadas como decisão no Quadro 4

1. Índices do B4 no 01_ddl.sql. Na seção de índices, remover `idx_venda_data_hora` e acrescentar os dois índices mantidos pelo B4:

```
-- remover:  CREATE INDEX idx_venda_data_hora ON venda (data_hora);
CREATE INDEX idx_venda_periodo_cobre
    ON venda (data_hora, status, id_funcionario, valor_total);
CREATE INDEX idx_item_venda_produto_qtd
    ON item_venda (id_produto, quantidade);
```

2. Auditoria identifica o usuário real. Nos dois gatilhos, trocar `CURRENT_USER()` por `USER()` nos três INSERT INTO auditoria (dois no gatilho de preço e um no de estoque). Em banco existente: DROP TRIGGER dos dois e recriação.

3. Fidelidade conforme a RN20. Em `CRUDVenda.creditarPontos`, trocar o cálculo de 1 ponto por real por 1 ponto a cada R$ 100,00:

```
// antes:  int pontos = total.setScale(0, RoundingMode.DOWN).intValue();
int pontos = total.divideToIntegralValue(new BigDecimal("100")).intValue();
```

4. Preços em DECIMAL(8,2). No 01_ddl.sql, trocar DECIMAL(5,2) por DECIMAL(8,2) em `produto.preco_custo`, `produto.preco_venda` e `item_venda.preco_congelado`. Em banco existente, testar o ALTER TABLE ... MODIFY (a restrição ck_produto_precos referencia essas colunas) ou recriar o banco com o script.

5. Credenciais fora do repositório. Criar o .gitignore e retirar do índice os arquivos locais:

```
printf "ToyStoreySystem/config.properties\nToyStoreySystem/build/\nToyStoreySystem/nbproject/private/\n" >> .gitignore
git rm --cached ToyStoreySystem/config.properties
git rm -r --cached ToyStoreySystem/build ToyStoreySystem/nbproject/private
git add .gitignore && git commit -m "Remove arquivos locais do versionamento"
```

O histórico do Git ainda guarda as senhas antigas; como são credenciais de laboratório, basta trocá-las no 05_acesso.sql e no config.properties local se forem reaproveitadas.

6. RN05: cliente com vendas não pode ser excluído. Trocar a ação da FK e tratar o erro na tela. Testado em uma cópia restaurada: a exclusão de uma pessoa cliente com vendas passou a falhar com o erro 1451 e o registro permaneceu.

```
ALTER TABLE venda DROP FOREIGN KEY fk_venda_cliente;
ALTER TABLE venda ADD CONSTRAINT fk_venda_cliente
    FOREIGN KEY (id_cliente) REFERENCES cliente (id_pessoa)
    ON DELETE RESTRICT ON UPDATE CASCADE;
-- na tela: se o erro for 1451, orientar “use Inativar”; considere ainda REVOKE DELETE
-- ON pessoa/cliente de app_gerente.
```

7. README. Corrigir os nomes dos arquivos que divergem do que está em docs/ (normalizacao.pdf, Transacoes_e_Concorrencia.pdf, apresentacao_04_10.pptx) e acrescentar, em “Como executar”, os scripts 03 a 05, o passo do config.properties e a execução do GeradorSenhas.